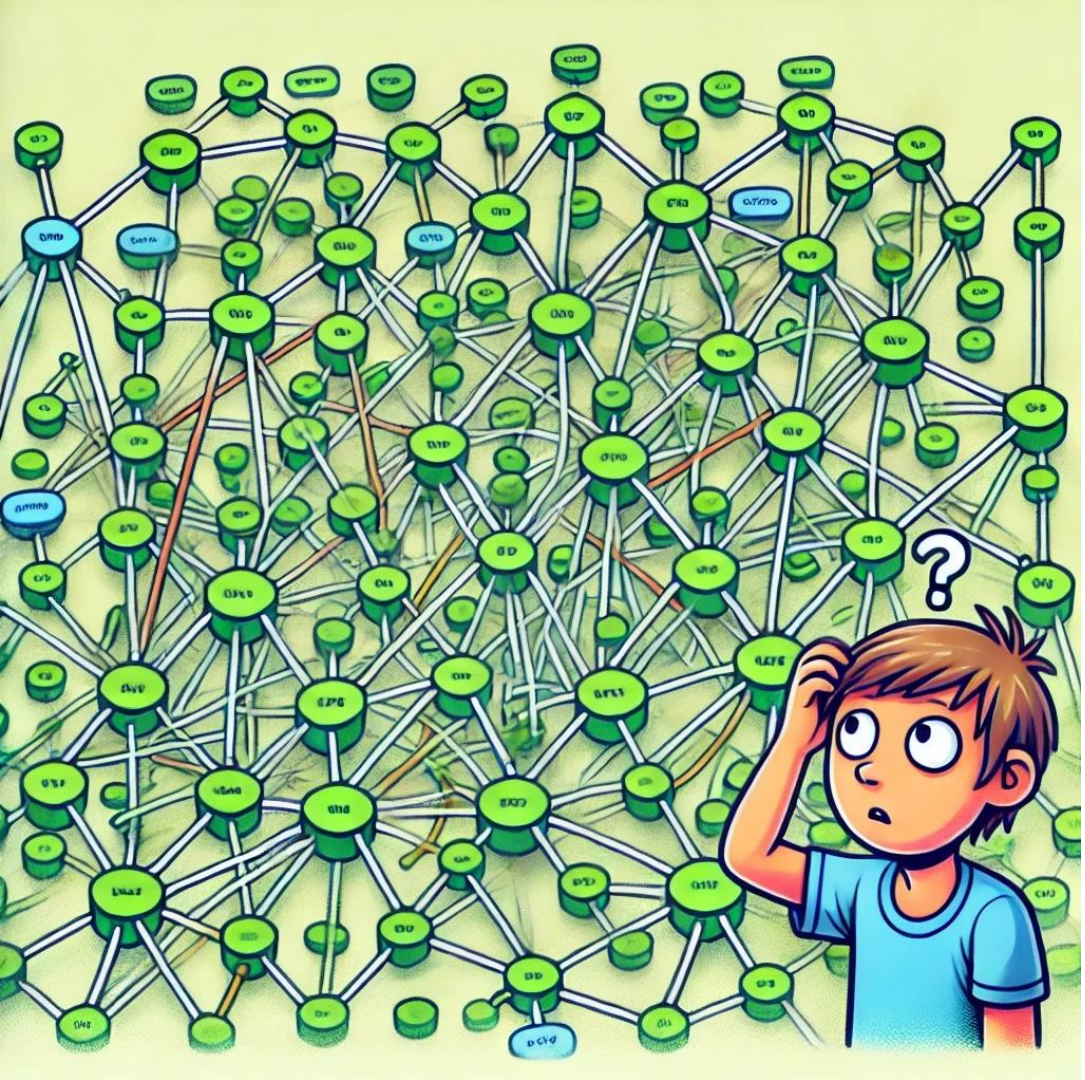
Test selection at Adyen

Saving time and resources

Maurício Aniche
Testing Enablement
Adyen

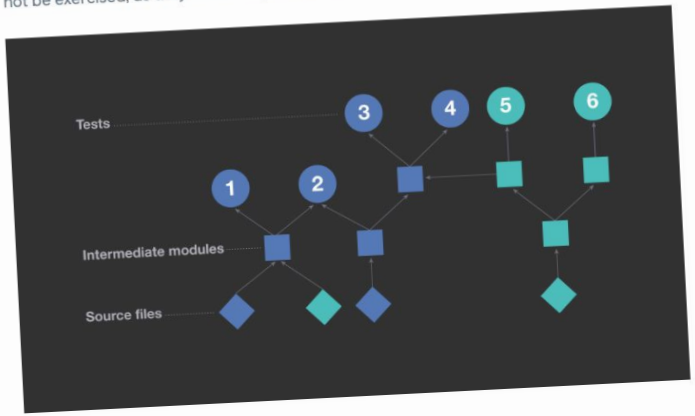
adyen



Letting your build tool decide which tests to run **might not be the most optimal idea**!

Why using build dependencies is inefficient

A common approach to regression testing is to use information extracted from build metadata to determine which tests to run on a particular code change. By analyzing build dependencies between units of code, one can determine all tests that transitively depend on sources modified in that code change. For example, in the diagram below, circles represent tests; squares represent intermediate units of code, such as libraries; and diamonds represent individual source files in the repository. An arrow connects entities A → B if and only if B directly depends on A, which we interpret as A impacting B. The blue diamonds represent two files modified in a sample code change. All entities transitively dependent upon them are also shown in blue. In this scenario, a test selection strategy based on build dependencies would exercise tests 1, 2, 3, and 4. But tests 5 and 6 would not be exercised, as they do not depend on modified files.



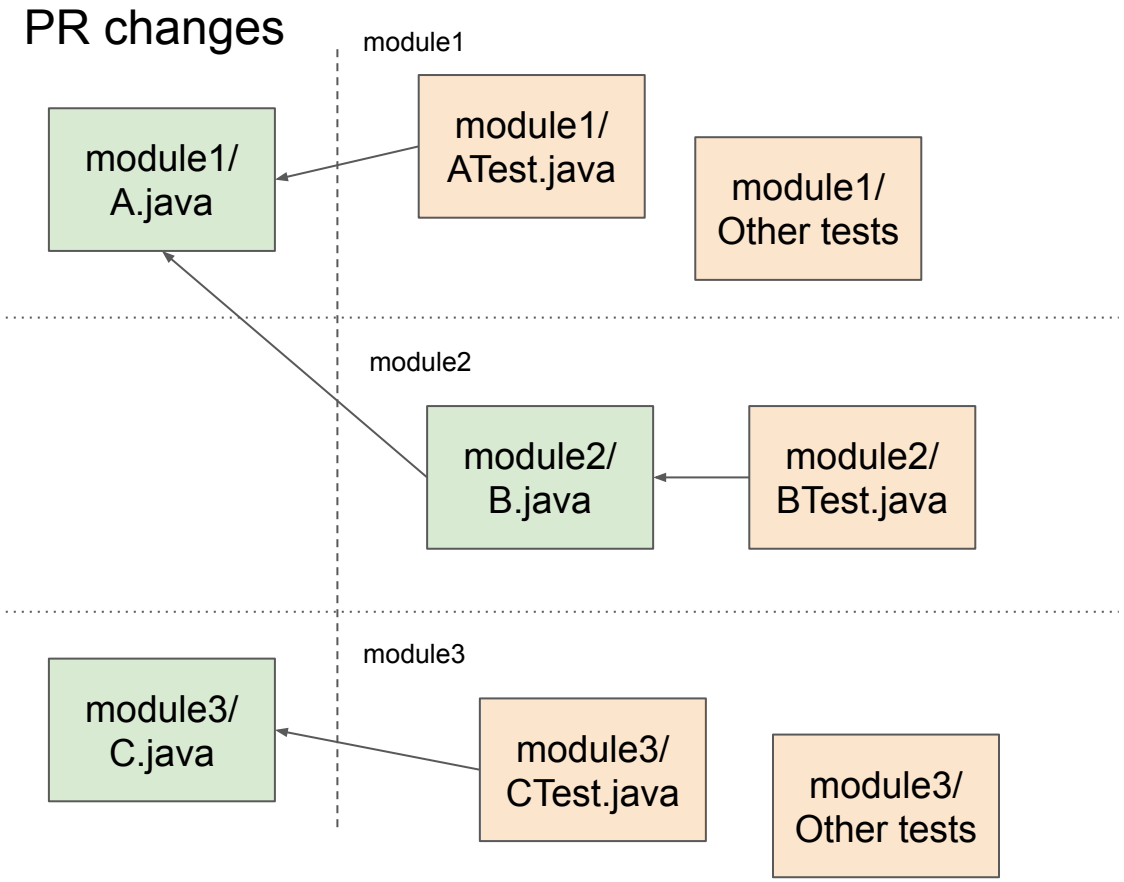
This approach has a significant shortcoming: It ends up saying "yes, this test is impacted" more often than is actually necessary. On average, it would cause as many as a quarter of all available tests to be exercised for each change made to our mobile codebase. If all

no alternative but to exercise each of them. However, in our monolithic codebase, end products depend on many reusable components, which use a small set of low-level libraries. In practice, many transitive dependencies are not, in fact, relevant for regression testing. For example, when there is a change to one of our low-level libraries, it would be inefficient to rerun all tests on every project that uses that library.

This is a **known problem** in large monorepos!

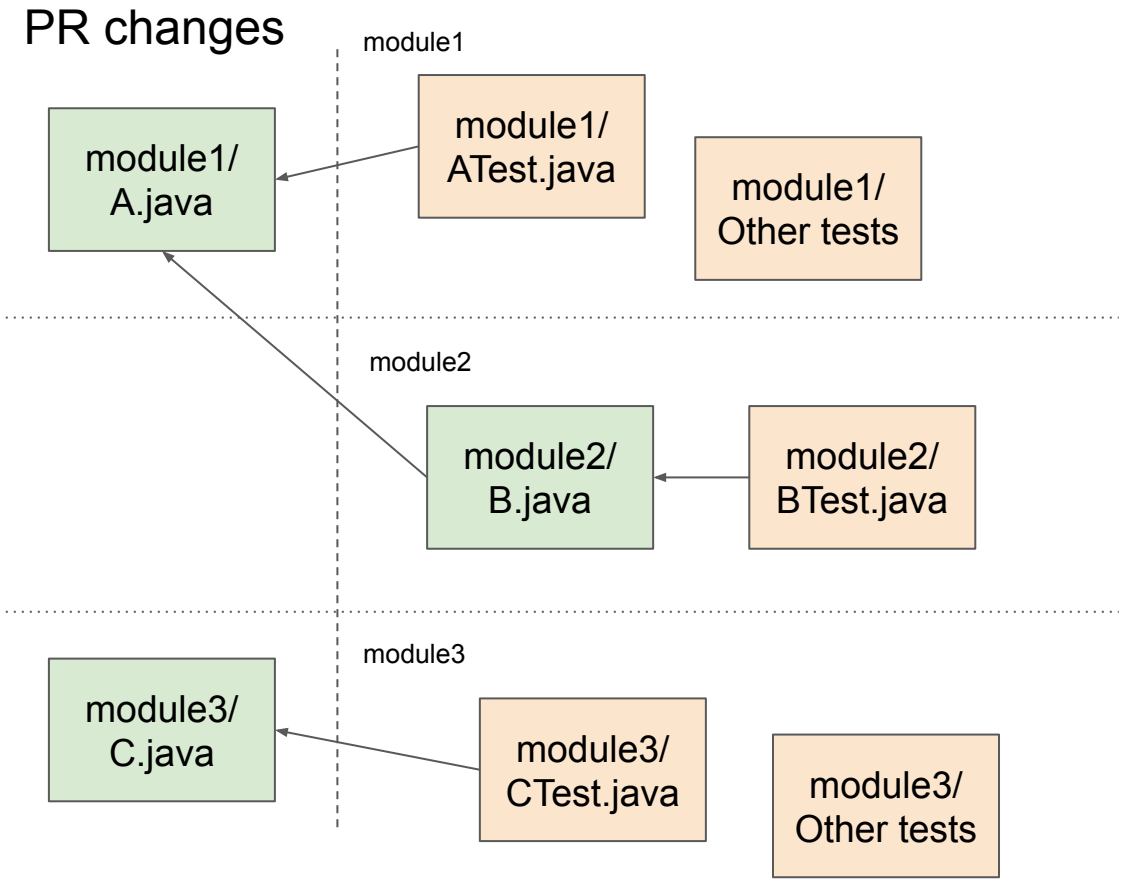
https://engineering.fb.com/2018/11/21/developer-tools/predictive-test-selection/

What if you could select, amongst all your tests, the ones that would **truly catch a regression** in the changed code?



Without test selection:

- **All** tests from module1
  - Module 1 changed
- **All** tests from module2
  - Module 2 depends on module 1
- **All** tests from module3
  - Module 3 changed



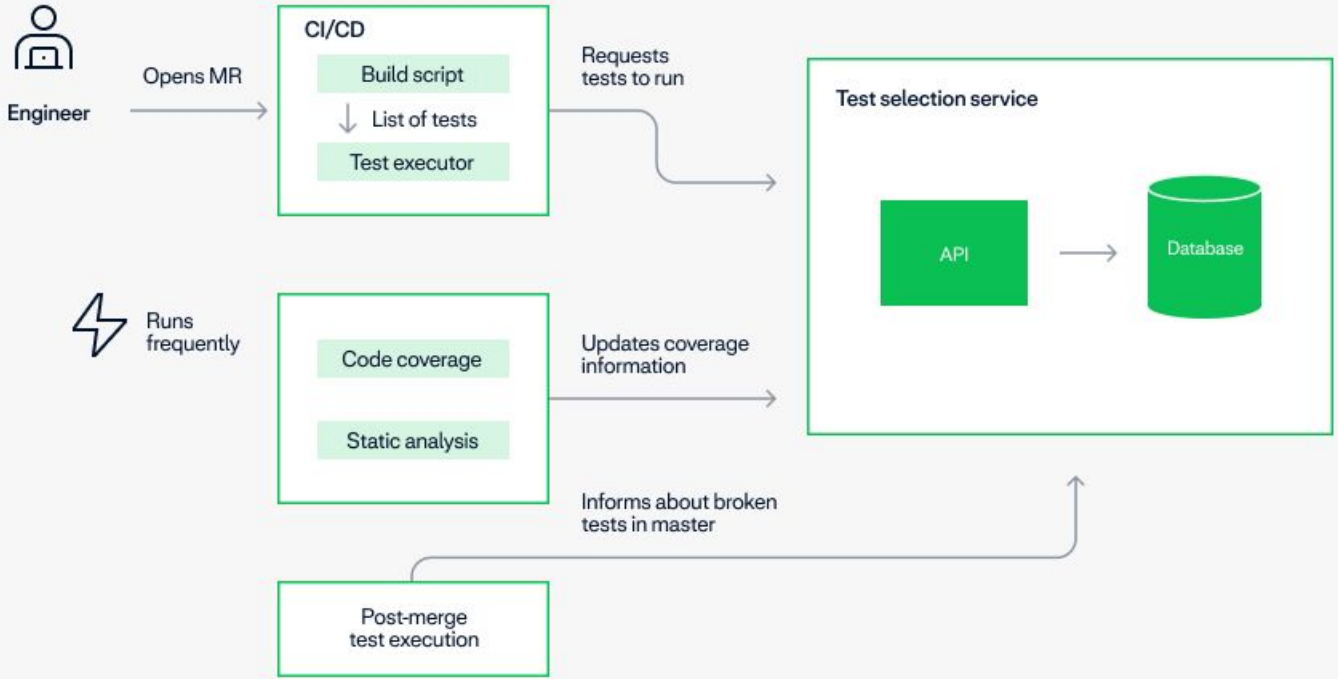
With test selection:

- ATest
- BTest
- CTest
- That's it!

Benefits

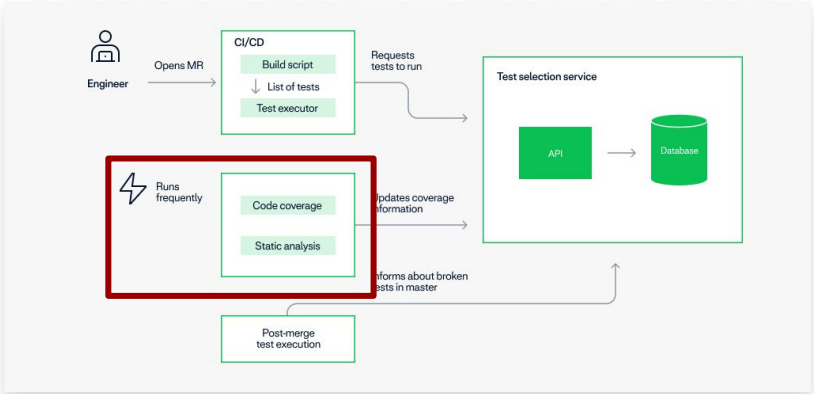
- 90%+ of pipelines run a subset of tests
- 30x reduction in test execution
- 12x reduction in CPU time
- Overall build pipelines 10%-30% faster

How did we **build it**, you may ask?


Engineer
Opens MR
CI/CD
Build script
List of tests
Test executor
Requests tests to run
Test selection service
API
Database
Runs frequently
Code coverage
Static analysis
Updates coverage information
Informs about broken tests in master
Post-merge test execution

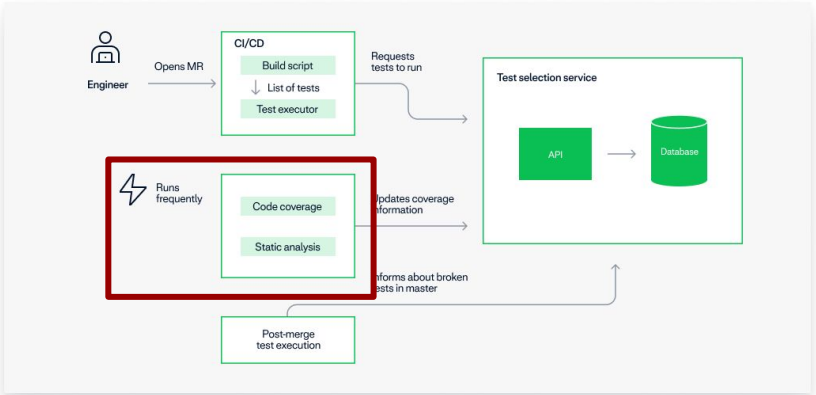
Code coverage

- Post-merge job that runs often and **collects fresh code coverage**
- JUnit listener that extracts class-level coverage information directly from Jacoco
  - Jacoco supports JMX
  - This one finding was key!
- POSTs the coverage data to the API
- Decision: class or method level?



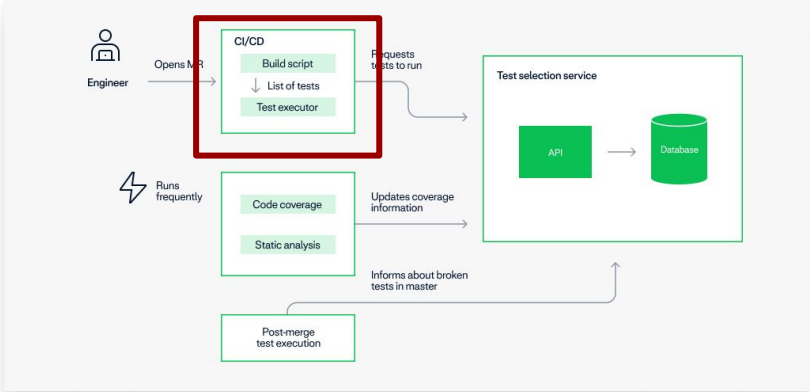
Static analysis

- **Coverage isn't enough**, as not everything generates coverage
  - Enums are a good example
  - Mybatis XMLs
  - Other in-house framework configuration files
- Async job that:
  - Clones the source code
  - Analyzes files in the repository (via simple parsing up to JavaParser proper analyzers)
  - POST data to test selection
- Links resource->code, code->code, test->code



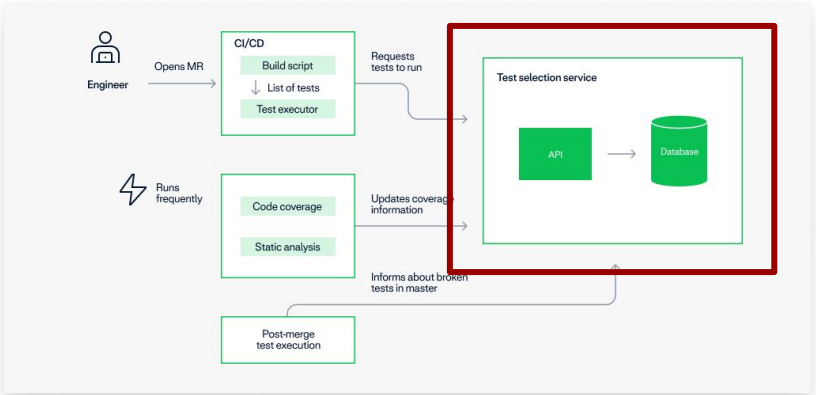
Build / CI

- Simple Python script that performs a GET request to our TS.
  - Input: list of modified files in the PR
  - Output: list of tests to run
- Gradle plugin that reads the test selection file and configure which tests to run
  - If no tests in a module, disable the test module entirely
  - Filter out the tests that are not in the list



Test selection API

- Background jobs updating coverage information that's coming from the sensors
- Selection API that returns the list of tests to run for a given set of changes
- Not a lot of magic here:
  - Spring Boot
  - Spring JPA
  - Lombok
  - Postgres
  - Caffeine



Test selection algorithm

- Request comes through a series of **Strategies**
- An strategy responds:
  - A set of specific tests to run
  - A set of modules to run all the tests
- A strategy can force the build to fallback to Gradle and run all the tests
  - Important for cases where we can see the prediction isn't good enough
- Examples:
  - CoverageStrategy, MybatisStrategy, …

```
public interface SelectionStrategy {  18 implementations  mauricio *
    /**
     * Returns the list of specific tests and/or modules to run all the tests
     * based on the heuristic of the implemented strategy.
     *
     * @param request The test selection request object, containing the list of files changed
     * @param configuration The selection configuration that's globally enabled for the service
     * @return A {@code SelectionStrategyOutcome} containing the list of tests and modules to run tests,
     *     as well as whether we should fall back to Gradle conservative test selection.
     */
    SelectionStrategyOutcome select(TestSelectionRequest request, SelectionConfiguration configuration);  18 implementations

    /**
     * Returns whether this strategy should be applied for the given request.
     *
     * @param request The test selection request
     * @return boolean indicating whether to run (true) or not (false)
     */
    default boolean accepts(TestSelectionRequest request) { return true; }
}
```

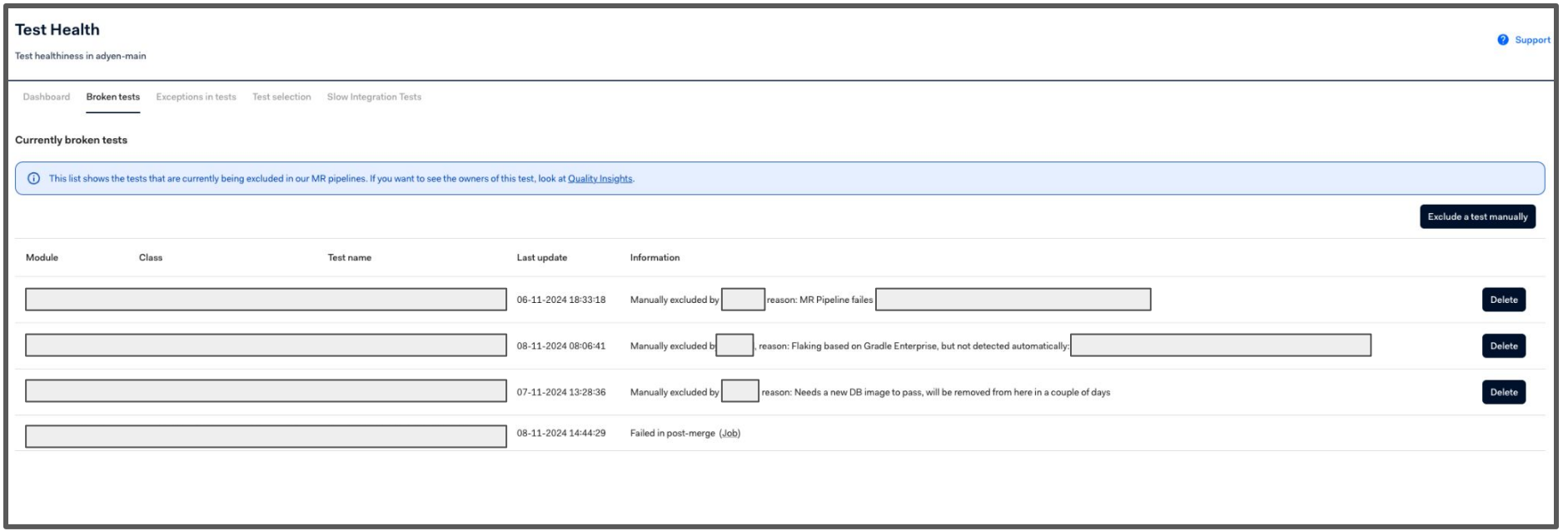
Interesting strategies

- Code coverage
- ResourceToProd, ProdToProd, and ProdToTest
- Modified modules
- DB migrations
- Exclusion List
- … and lots of Adyen-specific rules

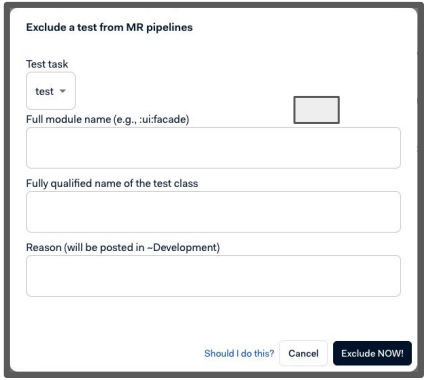
```java
public interface SelectionStrategy {  18 implementations  mauricio *
    /**
     * Returns the list of specific tests and/or modules to run all the tests
     * based on the heuristic of the implemented strategy.
     *
     * @param request The test selection request object, containing the list of files changed
     * @param configuration The selection configuration that's globally enabled for the service
     * @return A {@code SelectionStrategyOutcome} containing the list of tests and modules to run tests,
     *     as well as whether we should fall back to Gradle conservative test selection.
     */
    SelectionStrategyOutcome select(TestSelectionRequest request, SelectionConfiguration configuration);  18 implementations

    /**
     * Returns whether this strategy should be applied for the given request.
     *
     * @param request The test selection request
     * @return boolean indicating whether to run (true) or not (false)
     */
    default boolean accepts(TestSelectionRequest request) { return true; }
}
```

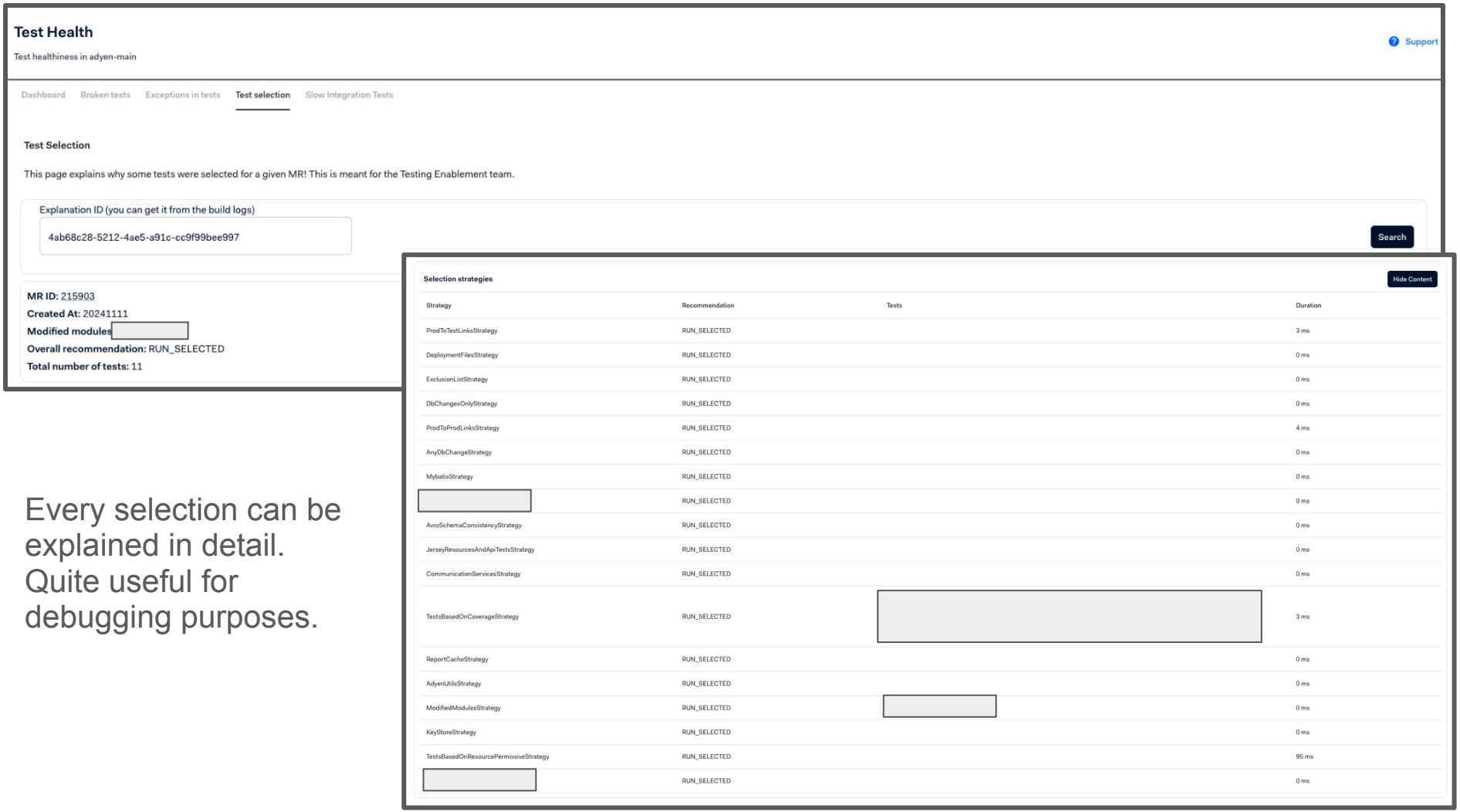
Broken tests

- It's **unavoidable** that broken tests will slip to master, for one reason or another
- We **monitor** for broken tests
  - Post-merge
  - Frequent breakages in the PRs
- They get **excluded automatically** from the next pipelines
- Authors are **notified** an the test is removed from the list when they start to pass again
- Ability to **manually exclude** a test
- **Saves thousands** of pipelines from breaking every month!



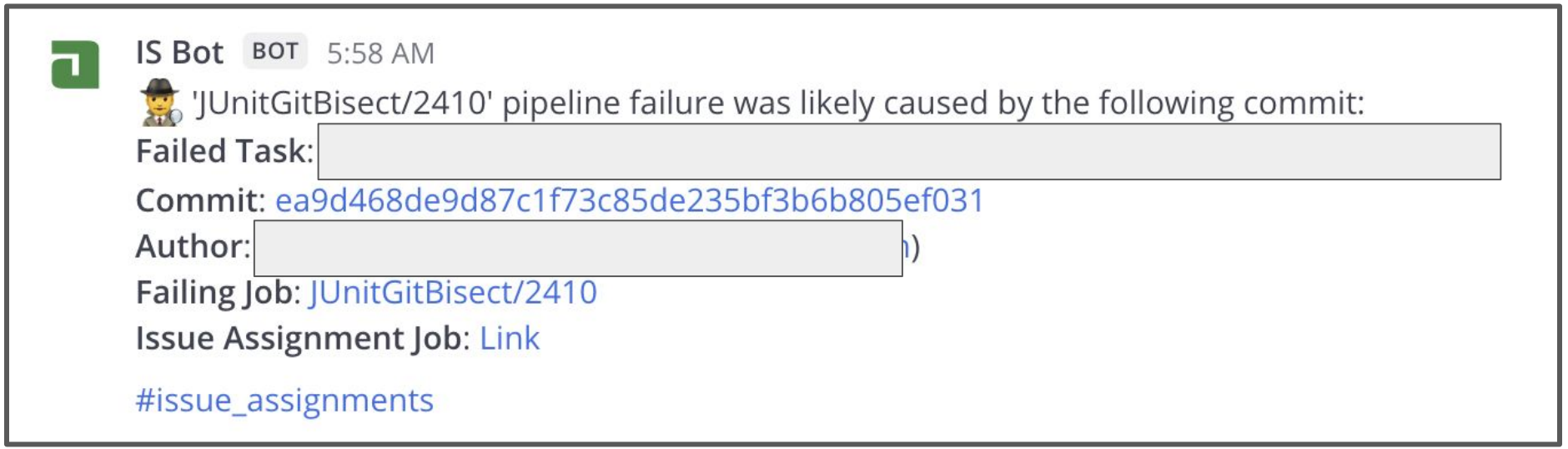
Everything can be found in our Launchpad (Adyen's Backstage) page!





Every selection can be explained in detail. Quite useful for debugging purposes.

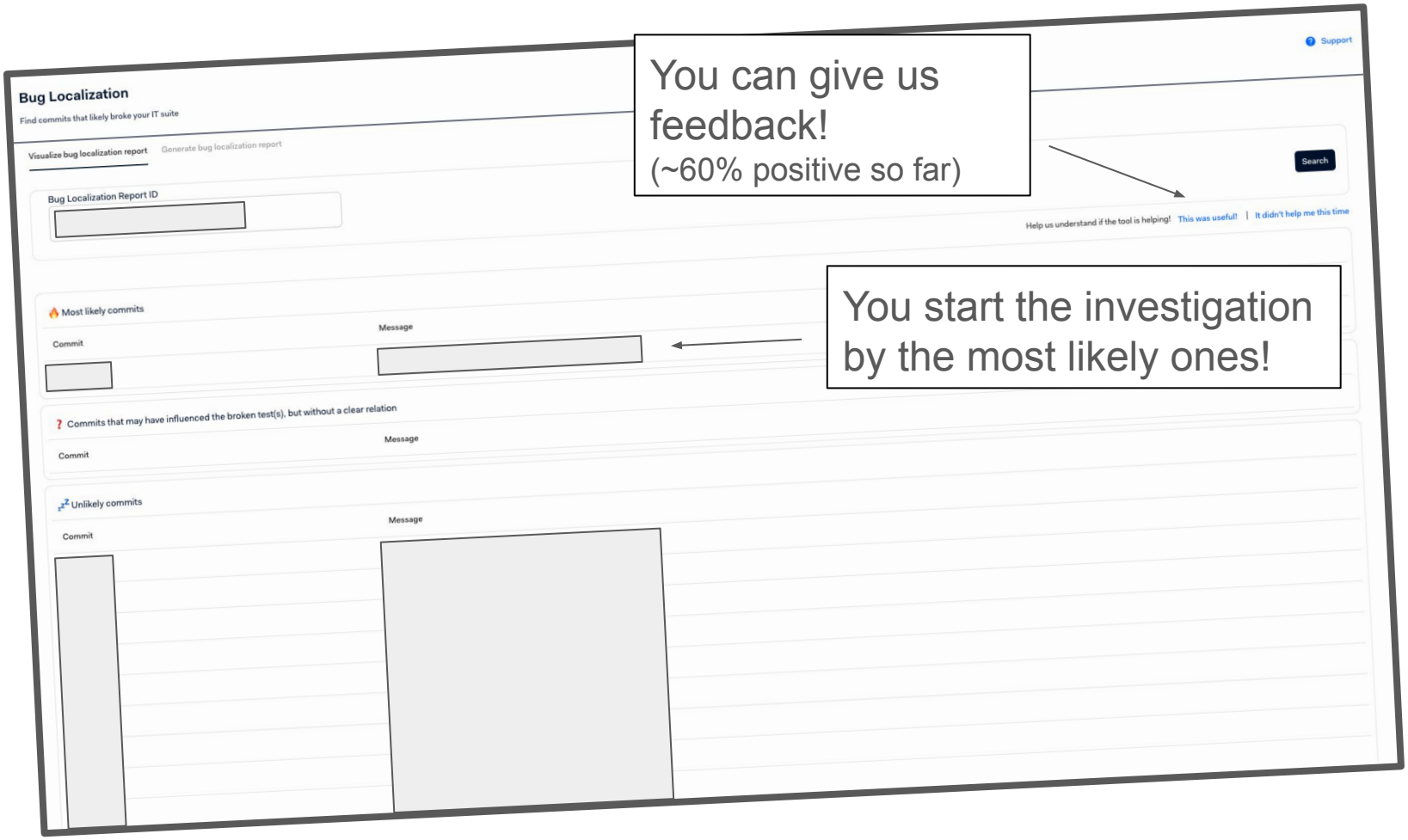
Finding the commit that broke the test… automatically!

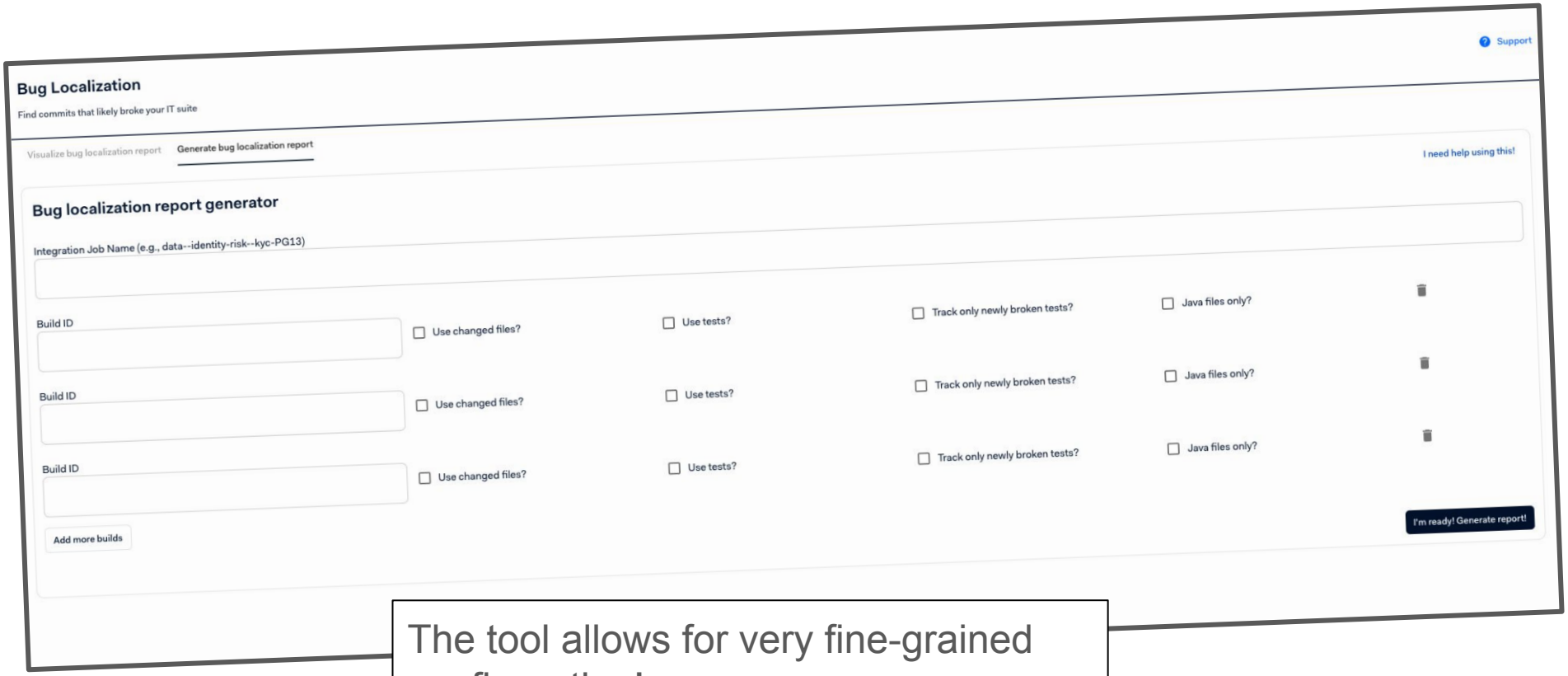


Automated Git Bisect to help find which commit broke the master!

Bug localization

- Identifying which commit broke the IT test is hard.
  - Too many commits in between
  - Too many webapps interacting
- "Test selection in reverse"
- For each commit in the range:
  - If the commit touches any file that is directly covered by the broken test, mark it as **red**
  - Not covered at all, mark it as **green**
  - In doubt, mark it as **yellow**
- A prioritized list of commits for engineers to start their investigation!


Bug Localization
Find commits that likely broke your IT suite
Support
Visualize bug localization report
Generate bug localization report
Search
Bug Localization Report ID
Help us understand if the tool is helping! This was useful! | It didn't help me this time
Most likely commits
Commit
Message
Commits that may have influenced the broken test(s), but without a clear relation
Commit
Message
Unlikely commits
Commit
Message
You can give us feedback!
(~60% positive so far)
You start the investigation by the most likely ones!



The tool allows for very fine-grained configuration!

Lessons learned and what's next

- You won't have full or precise coverage information
- Have an exclusion list ready for quick fixes
- Precision versus simplicity
- Get ready to handle broken tests
- Resilience, reliability, and monitoring

- Future: test selection for integration tests

adyen Products Businesses we serve About Resources Pricing

Log in

← Knowledge Hub

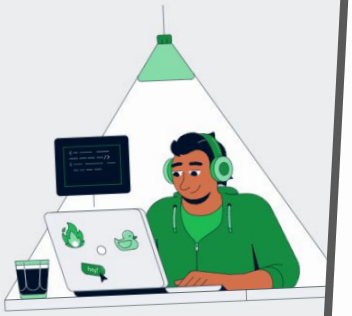
https://www.adyen.com/knowledge-hub/test-selection-at-adyen

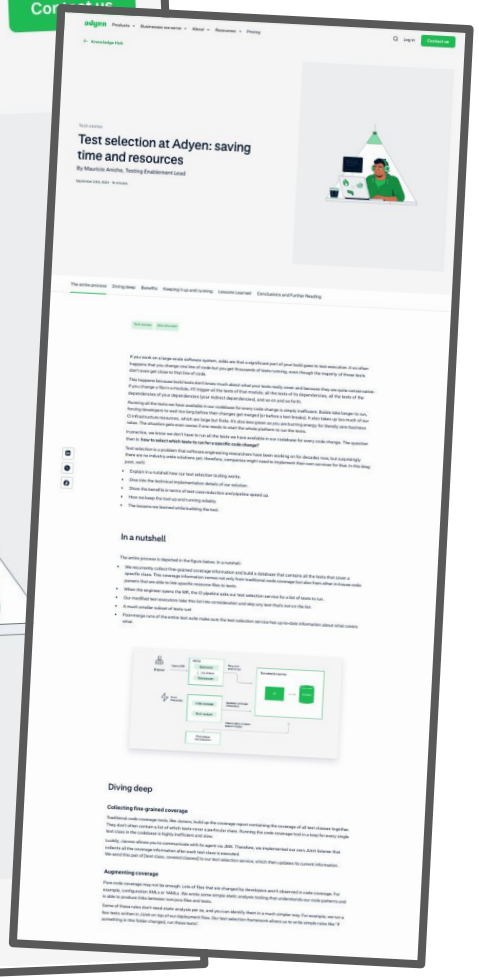
Tech stories

Test selection at Adyen: saving time and resources

By Mauricio Aniche, Testing Enablement Lead

September 23rd, 2024 · 15 minutes





Testing Enablement Team @ Adyen

Maurício Aniche

Dmytro Zaitsev

Vadym Zakharchuk

Bhathiya Jayasekara

Ali Kanso

https://www.adyen.com/knowledge-hub/test-selection-at-adyen

@mauricioaniche
mauricio.aniche@adyen.com
https://www.mauricioaniche.com

adyen